

Laboratorio di Making - Report

Rosario Zaccone

Indice

1	Idea	2
2	Analisi di Mercato	4
2.1	Stima dei costi	4
2.2	Comparazione con altri prodotti	4
3	Funzionalità	6
4	Design e implementazione	6
4.1	Design hardware	6
4.2	Design software	8
4.2.1	Architettura generale	8
4.3	Architettura di implementazione	9
4.3.1	Algoritmo di rilevazione corpi estranei	11
4.3.2	Algoritmo di autocalibrazione	11
4.4	Comandi bot Telegram	12
5	Risultati	14
5.1	Calibrazione	14
5.2	Configurazione	15
5.3	Timelapse	16
5.4	Rilevamento corpi estranei	18
6	Conclusioni	24

1 Idea

L'idea del progetto nasce dalle seguenti necessità:

- irrigare autonomamente e a distanza le proprie piante
- tenere sott'occhio costantemente le proprie piante, generando timelapse utili per monitorare lo stato di crescita, ma anche per puro svago
- individuare i corpi estranei che vengono a contatto con le piante, e identificarli. Avere un diario di insetti (o altri invertebrati) che vengono a contatto con le piante è utile, oltre per puro scopo naturalistico, per avere indicazioni sul loro stato di salute

Da queste necessità nasce l'idea di sviluppare *Clorofillo*, un sistema in grado di assistere fino a tre vasi con piante, gestendo l'irrigazione, realizzando timelapse della crescita e individuando possibili corpi estranei, con la possibilità di identificarli.

Il sistema comprende un Raspberry Pi, con i relativi componenti e sensori, e un bot Telegram che consente di configurarlo, sfruttare le funzionalità offerte e ricevere notifiche in tempo reale. Al Raspberry Pi sono collegati sensori di umidità del suolo e pompe per la gestione dell'irrigazione automatica; il sistema include inoltre un sensore di livello dell'acqua e un serbatoio, utilizzati per notificare l'utente quando il livello dell'acqua è insufficiente. È presente un convertitore analogico-digitale per trasformare il segnale analogico dei sensori in un formato digitale utilizzabile dal Raspberry Pi.



Figura 1: Bozzetto del sistema

È inoltre presente una Pi Camera, montata su un braccio collegato a un servo motore, che consente la rotazione della fotocamera e il supporto fino a tre vasi. Le posizioni dei vasi vengono registrate, in modo che il sistema sappia dove posizionare la fotocamera per scattare le immagini necessarie alla realizzazione dei timelapse e alle altre funzionalità. Tali posizioni possono essere configurate manualmente dall'utente oppure individuate automaticamente dal sistema tramite una procedura di autocalibrazione basata su codici ArUco posizionati sui vasi.

L'individuazione dei corpi estranei si basa sulla seguente ipotesi: date due immagini della stessa pianta in vaso, scattate a pochi secondi di distanza, assumendo l'assenza di vento, di variazioni improvvise di illuminazione e di interazioni umane con la pianta, la presenza di una differenza significativa tra le due immagini indica l'interazione di un corpo estraneo, come un invertebrato o un altro animale, oppure il verificarsi di eventi come la caduta di una foglia, di un fiore o di un frutto. Poiché le immagini vengono acquisite a distanza di pochi secondi, si escludono fenomeni lenti come la crescita o la fioritura della pianta, così come variazioni graduali della luce solare, che produrrebbero differenze significative solo su intervalli temporali più lunghi.

La rilevazione dei corpi estranei avviene nel seguente modo: a intervalli regolari vengono scattate immagini dello stesso vaso, e ogni immagine viene confrontata con la precedente mediante un algoritmo in grado di individuare differenze significative. In presenza di tali differenze, vengono estratte le regioni dell'immagine interessate (patch) e inviate

all'utente. L'utente può quindi scegliere se ignorare le patch, nel caso di falsi positivi, oppure salvarle aggiungendo una nota descrittiva sul contenuto, ad esempio il tipo di animale rilevato. È inoltre possibile procedere con l'identificazione automatica dell'insetto o dell'animale tramite l'utilizzo di API esterne.

Nella Figura 1 è mostrato un bozzetto del sistema, nella Figura 2 la sua implementazione reale.



Figura 2: Clorofillo

2 Analisi di Mercato

2.1 Stima dei costi

Per avere un'idea dei costi necessari per la realizzazione del progetto, nella Tabella 1 sono elencati i componenti principali, la quantità prevista e il costo stimato per ciascuno.

Come si può osservare dalla Tabella 1, i componenti principali più costosi sono il Raspberry Pi 4 e l'alimentatore, mentre sensori e piccoli accessori hanno un impatto economico contenuto.

2.2 Comparazione con altri prodotti

Nella Tabella 2 è riassunta una comparazione con prodotti simili.

Componente	Quantità	Costo stimato (EUR)
Sensore umidità suolo	3	3
Pompa acqua 5V	3	5
Sensore livello acqua	1	1
Relay 4 channel	1	4
Alimentatore 5V 2A	1	7
Servo motore MG995	1	5
Raspberry Pi 4	1	55
Jumper, cavi vari	-	5
Scatola contenimento	1	10
Serbatoio acqua	1	2
Tubi per irrigazione	-	1
Convertitore analogico digitale MCP3008	1	2
Totale stimato	-	100 EUR

Tabella 1: Stima dei costi dei componenti del progetto

Prodotto	Funzionalità principali	Prezzo stimato	Confronto con Clorofillo
PLANTSIO	Vaso smart con sensori di umidità, luce e temperatura, display, irrigazione base. App mobile per configurazione e notifiche.	80–100 €	Supporta un solo vaso di dimensione fissa; irrigazione non personalizzabile.
PlantCam	Fotocamera timelapse per piante; acquisizione immagini periodiche. App mobile per configurazione e notifiche.	50–60 €	La fotocamera non è rotante, quindi si concentra solamente su un solo soggetto; non individua corpi estranei.
pi-timelapse	Programma desktop che permette di creare timelapse personalizzabile con Raspberry Pi; dà la possibilità di creare GIF e video. Non è focalizzato sulle piante.	70–100 €	La fotocamera non è rotante, quindi si concentra solamente su un solo soggetto; non individua corpi estranei. Permette più personalizzazione dal punto di vista della generazione del timelapse (imaging). Interfaccia tramite CLI.

Tabella 2: Comparazione di Clorofillo con prodotti simili sul mercato.

Clorofillo supera i limiti comuni dei prodotti analizzati, che risultano focalizzati su una singola funzione o su un singolo soggetto. PLANTSIO offre un sistema di irrigazione e monitoraggio limitato a un solo vaso, senza supporto al monitoraggio visivo. PlantCam consente l’osservazione della crescita tramite timelapse, ma è vincolata a un unico soggetto e non integra funzionalità di analisi o intervento. Il progetto open-source *pi-timelapse*, pur garantendo un’elevata personalizzazione nella generazione di contenuti visivi, non è progettato specificamente per il contesto botanico, non presenta funzionalità di rilevazione di corpi estranei e interfacciarsi con esso è più complicato: non dispone di app o

bot Telegram, quindi l'utente deve essere pratico di utilizzo di CLI. Clorofillo si distingue integrando in un unico sistema il supporto a più vasi tramite una fotocamera rotante, il monitoraggio visivo continuo con timelapse, l'irrigazione automatica personalizzata e la rilevazione di corpi estranei. A queste funzionalità si aggiunge un bot Telegram, che consente all'utente di interfacciarsi in modo semplice e immediato con il sistema, permettendo il controllo in tempo reale, la ricezione di notifiche e la consultazione dello stato delle piante da remoto. In questo modo Clorofillo non si limita all'osservazione passiva, ma introduce un approccio attivo, intelligente e accessibile alla cura delle piante, colmando il divario tra soluzioni consumer limitate e progetti altamente tecnici ma poco accessibili.

3 Funzionalità

Di seguito si elencano le funzionalità del sistema:

- Configurazione dei seguenti parametri: dimensione vaso, nome piante, attivazione irrigazione, periodi in cui scattare foto per timelapse, attivazione rilevazione corpi estranei, frequenza scatti per individuazione corpi estranei, posizione vasi, soglia di umidità per irrigazione
- Autocalibrazione della posizione dei vasi, tramite rilevamento codici Aruco
- Irrigazione automatica basata sulla dimensione del vaso e sui livelli di umidità del suolo
- Creazione di timelapse di uno specifico vaso, dati intervallo di tempo, fps e filtri
- Rilevazione corpi estranei
- Possibilità di registrare i corpi estranei nel sistema e scartare i falsi positivi
- Possibilità di identificare insetti e altri invertebrati
- Possibilità di creare un diario degli avvistamenti di uno specifico vaso
- Interazione con il sistema via bot Telegram
- Notifiche in tempo reale riguardo individuazione corpi estranei e serbatoio vuoto

4 Design e implementazione

4.1 Design hardware

Il design hardware del progetto è illustrato in Figura 3. Essa mostra la disposizione dei sensori, delle pompe, dei serbatoi e degli altri componenti all'interno del sistema. Il progetto prevede due alimentatori separati: uno dedicato ad alimentare il Raspberry Pi, garantendo stabilità e sicurezza per il microcontrollore, e un secondo che alimenta le pompe e il servo. La separazione è necessaria perché i motori e le pompe richiedono correnti più elevate che potrebbero interferire con l'elettronica sensibile del Raspberry.

Il servo è utilizzato per orientare la camera, che è montata sul suo braccio, permettendo di acquisire immagini dei vasi da diverse angolazioni. Un convertitore analogico-digitale

MCP3008 è presente per permettere al Raspberry Pi di leggere segnali analogici dai sensori, come il livello dell'acqua nei serbatoi e l'umidità del terreno, dati fondamentali per il monitoraggio e il controllo del sistema.

Infine, è integrato un relay che gestisce l'accensione e lo spegnimento delle pompe. Questo componente è necessario per isolare elettricamente il Raspberry Pi dai circuiti ad alta corrente delle pompe, garantendo sicurezza e protezione dei componenti elettronici e permettendo un controllo affidabile del flusso d'acqua nei vasi.

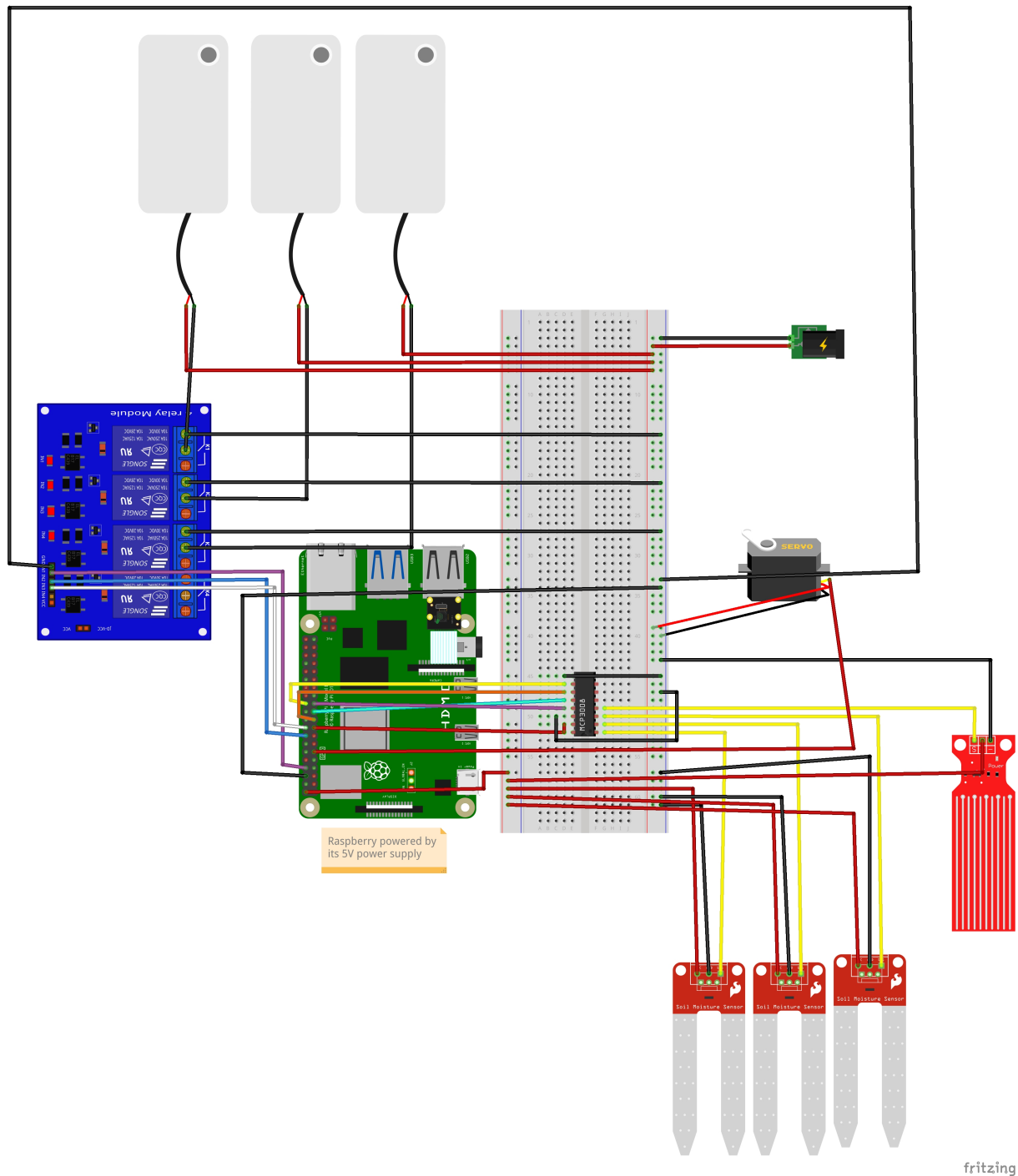


Figura 3: Design hardware del sistema

4.2 Design software

4.2.1 Architettura generale

L'architettura software del progetto è schematizzata in Figura 4. Essa evidenzia i principali moduli software e le interazioni tra loro, comprese le componenti di acquisizione dati, controllo dei vasi e gestione della comunicazione.

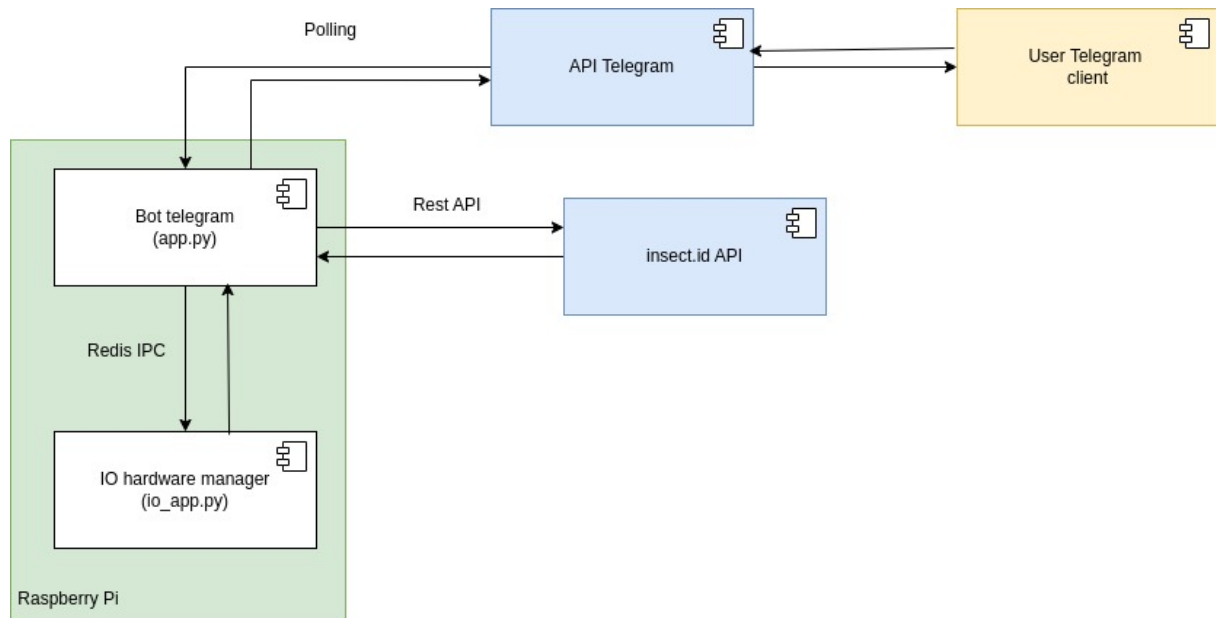


Figura 4: Architettura generale del software

Si elencano i componenti principali del sistema:

- **IO Hardware Manager:** processo Python che gestisce i sensori e gli altri componenti hardware collegati al sistema. Consiste in due thread: uno che si occupa dell'irrigazione, e uno che si occupa della parte visiva (timelapse e rilevazione corpi estranei). I due thread girano in parallelo.
- **Bot Telegram:** processo Python che implementa il bot Telegram e interroga periodicamente le API di Telegram tramite polling per ricevere i comandi dell'utente. Il bot consente la configurazione del sistema, l'invio di comandi ai moduli interni e la ricezione e l'inoltro di notifiche in tempo reale generate dal sistema.
- **API Telegram:** servizio esterno fornito da Telegram che consente al bot di inviare e ricevere messaggi, notifiche e aggiornamenti dagli utenti. Il bot interagisce con le API tramite polling per controllare lo stato dei messaggi e inviare risposte o notifiche in tempo reale.
- **insect.id API:** servizio esterno per identificazione insetti e altri invertebrati
- **User Telegram client:** classica applicazione mobile Telegram per usufruire del bot e interfacciarsi con il sistema

I due processi Python comunicano fra loro tramite Redis IPC, la comunicazione fra loro è necessaria per inviare notifiche riguardo il serbatoio dell'acqua e la rilevazione di corpi estranei (da IO hardware manager a Bot Telegram) e l'inizio della procedura di autocalibrazione e di salvataggio/identificazione corpi estranei (da Bot Telegram a IO).

4.3 Architettura di implementazione

Il software è basato su una classica architettura a strati, scelta per la sua semplicità. Per la persistenza dei dati è stato utilizzato SQLite, in quanto leggero, integrabile senza configurazione di un server dedicato e ideale per applicazioni locali o embedded come su Raspberry Pi, garantendo al contempo affidabilità e supporto per transazioni. Il filesystem viene invece impiegato per memorizzare le foto. Il diagramma dell'architettura è mostrato nella Figura 6, mentre il modello è mostrato nella Figura 5.

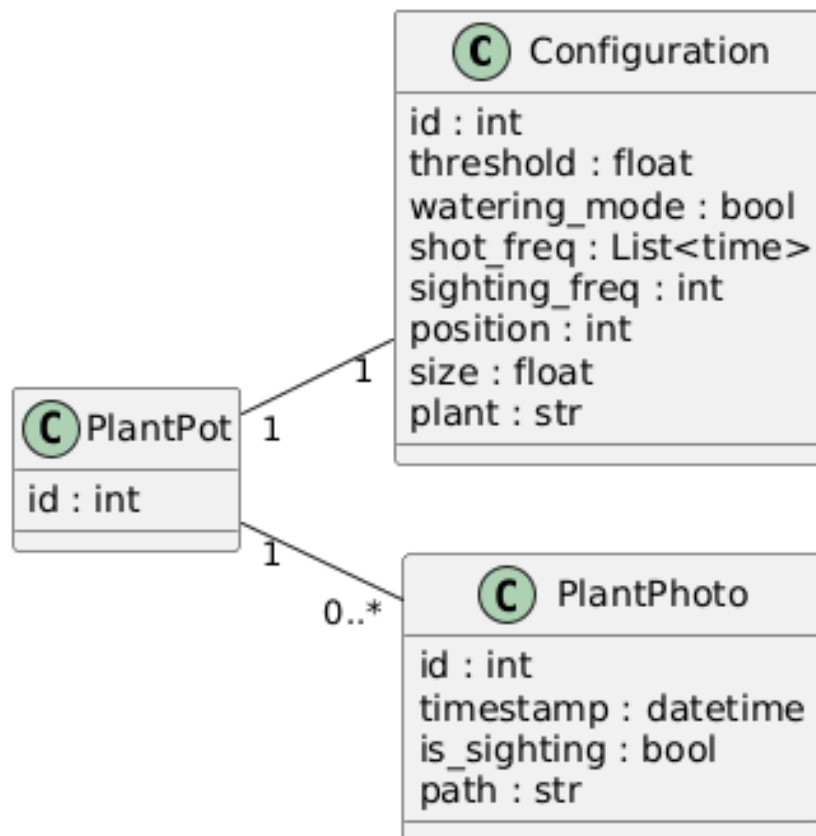


Figura 5: Diagramma del modello di dominio e delle entità principali.

Di seguito elencata la relativa implementazione del design proposto.

- **Persistence layer**

- **ConfigurationORM**: entità SQLAlchemy che rappresenta la configurazione di un vaso; mantiene parametri di irrigazione, rilevamento e posizione ed è associata uno-a-uno a un **PlantPotORM**.
- **ShotTimeORM**: entità che rappresenta un singolo orario di scatto; è associata a una sola **ConfigurationORM** (relazione uno-a-molti).
- **PlantPotORM**: entità persistente del vaso; possiede una configurazione unica e una collezione di foto.
- **PlantPhotoORM**: entità che rappresenta una foto acquisita dal sistema; ogni foto appartiene a un solo vaso.
- **Repository**: classe astratta che definisce l'interfaccia comune per l'accesso ai dati (CRUD).

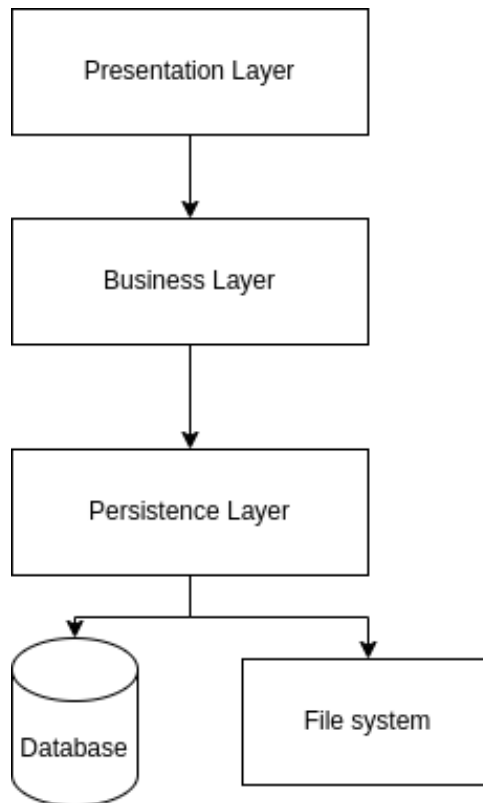


Figura 6: Diagramma dell'architettura a strati dell'applicazione.

- **PlantPhotoRepository**: repository concreto per la gestione delle foto, incluse operazioni di pulizia e query specifiche.
- **PlantPotRepository**: repository concreto per i vasi, con metodi di accesso avanzati alle foto per intervallo temporale.
- **ConfigurationRepository**: repository dedicato alla persistenza e all'aggiornamento delle configurazioni.

- **Business layer**

- **Configuration**: modello di dominio che incapsula la configurazione logica di un vaso, con validazione dei parametri e conversione da/verso ORM.
- **PlantPhoto**: modello di dominio di una foto, indipendente dalla persistenza, utilizzato nei servizi applicativi.
- **PlantPot**: modello di dominio che rappresenta un vaso e coordina configurazione e foto associate.
- **PlantPhotoService**: servizio applicativo che gestisce acquisizione immagini, rilevamento insetti e salvataggio delle foto.
- **PlantPotService**: servizio applicativo responsabile di timelapse, diario insetti e calibrazione del sistema.

- **Presentation layer**

- La parte di presentation è affidata al client Telegram e al relativo bot

4.3.1 Algoritmo di rilevazione corpi estranei

L'algoritmo `_detect_sighting_patches_base64` è progettato per rilevare eventuali corpi estranei (come insetti) presenti su vasi con piante confrontando due immagini consecutive: una "before" e una "after", scattate a pochi secondi di distanza. L'elaborazione avviene in più fasi:

1. **Caricamento e conversione in scala di grigi:** le due immagini vengono caricate e convertite in scala di grigi per semplificare l'elaborazione e ridurre il rumore cromatico.
2. **Allineamento delle immagini:** per compensare eventuali piccoli spostamenti della fotocamera, le immagini vengono allineate utilizzando keypoint ORB (Oriented FAST and Rotated BRIEF). Vengono calcolate corrispondenze tra punti caratteristici e stimata una trasformazione affine, in modo da minimizzare falsi positivi dovuti a movimenti della fotocamera.
3. **Calcolo della differenza:** l'immagine "after" viene sottratta dall'immagine "before" allineata per evidenziare le aree con cambiamenti. Si applica poi una soglia binaria per isolare le regioni più significative.
4. **Filtraggio morfologico:** operazioni di apertura e dilatazione rimuovono rumore e pixel isolati, consolidando le regioni rilevanti.
5. **Estrazione dei contorni:** vengono identificati i contorni delle aree modificate. Ogni contorno viene filtrato in base a criteri di area, dimensioni e distanza dai margini dell'immagine, per evitare falsi positivi o elementi fuori dal vaso.
6. **Espansione del contesto e salvataggio:** attorno a ciascun contorno valido viene aggiunto un margine extra (padding) per includere un contesto maggiore. Ogni patch viene salvata sul filesystem con un identificatore unico.
7. **Conversione in Base64:** le patch salvate vengono convertite in stringhe Base64, pronte per essere inviate a eventuali servizi esterni di riconoscimento o analisi.

4.3.2 Algoritmo di autocalibrazione

L'algoritmo di autocalibrazione consiste nei seguenti passi:

1. **Acquisizione delle foto nei diversi angoli:** la fotocamera ruota progressivamente e si posiziona a ogni angolazione compresa tra 0° e 180° , con incrementi di 10° , scattando una foto a ciascuna posizione.
2. **Ricerca ArUco:** nelle diverse foto vengono ricercati i codici ArUco, e vengono salvate le posizioni relative alle foto dove i codici sono centrati orizzontalmente. Nei codici ArUco sono codificati gli id dei vasi.
3. **Risultato finale:** se sono stati rilevati tutti e tre gli ArUco, le posizioni vengono salvate nella configurazione. Se qualche ArUco non viene rilevato, la configurazione non avviene e vengono comunicati gli id dei vasi non rilevati. In entrambi i casi vengono inviati all'utente, via bot Telegram, le foto scattate in tutti gli angoli, in modo che l'utente possa capire come posizionare meglio i vasi per la calibrazione in caso di fallimento.

4.4 Comandi bot Telegram

/start

Parametri: -

Descrizione: Autorizza la chat e abilita i comandi.

Esempio: `/start`

/help

Parametri: -

Descrizione: Alias di `/start`, mostra guida rapida.

Esempio: `/help`

/info

Parametri: -

Descrizione: Mostra tutti i comandi e significato dei parametri.

Esempio: `/info`

/settings

Parametri: `<pot_id>`

Descrizione: Mostra la configurazione del vaso.

Esempio: `/settings 1`

/setsettings

Parametri: `<id>` `<watering_mode>` `<threshold>` `<shot_freq>` `<sighting_freq>` `<position>` `<plant>` `<size>`

Descrizione: Aggiorna tutti i parametri di configurazione di un vaso.

Esempio: `/setsettings 1 true 45.5 08:00,12:00 2 90 Basilico 3.5`

/set_threshold

Parametri: `<pot_id>` `<threshold>`

Descrizione: Imposta la soglia di umidità (%).

Esempio: `/set_threshold 1 45.5`

/set_watering_mode

Parametri: `<pot_id>` `<true|false>`

Descrizione: Abilita/disabilita irrigazione automatica.

Esempio: `/set_watering_mode 1 true`

/set_shot_freq

Parametri: `<pot_id>` `<HH:MM,HH:MM,...>`

Descrizione: Imposta orari timelapse.

Esempio: `/set_shot_freq 1 08:00,12:00,18:00`

/set_sighting_freq

Parametri: `<pot_id>` `<frequency>`

Descrizione: Frequenza rilevamento corpi estranei (shots/min).

Esempio: `/set_sighting_freq 1 2`

/set_position

Parametri: `<pot_id>` `<position>`

Descrizione: Posizione del vaso (0–180°).

Esempio: `/set_position 1 90`

/set_size

Parametri: `<pot_id>` `<size>`

Descrizione: Capacità del vaso in litri.

Esempio: `/set_size 1 3.5`

`/set_plant`

Parametri: `<pot_id> <plant_name>`

Descrizione: Nome della pianta in vaso.

Esempio: `/set_plant 1 Basilico`

`/timelapse`

Parametri: `<pot_id> <from YYYY-MM-DD> <to YYYY-MM-DD> <fps> <filter>`

Descrizione: Genera un video timelapse del vaso.

Esempio: `/timelapse 1 2025-01-01 2025-01-31 24 bw`

`/calibrate`

Parametri: -

Descrizione: Avvia autocalibrazione per settare le posizioni dei vasi.

Esempio: `/calibrate`

`/diary`

Parametri: `<pot_id>`

Descrizione: Genera diario degli avvistamenti di corpi estranei.

Esempio: `/diary 1`

`/cleansighting`

Parametri: -

Descrizione: Elimina foto degli avvistamenti da DB e filesystem.

Esempio: `/cleansighting`

`/cleantimelapse`

Parametri: -

Descrizione: Elimina foto timelapse da DB e filesystem.

Esempio: `/cleantimelapse`

`/shutdown`

Parametri: -

Descrizione: Spegne il Raspberry Pi.

Esempio: `/shutdown`

`/killio`

Parametri: -

Descrizione: Termina il processo IO Hardware manager.

Esempio: `/killio`

`/startio`

Parametri: -

Descrizione: Avvia il processo IO Hardware manager.

Esempio: `/startio`

`/savesighting`

Parametri: (reply a foto) [description]

Descrizione: Salva una foto del corpo estraneo avvistato con descrizione opzionale.

Esempio: `/savesighting ape`

`/identifyinvertebrate`

Parametri: (reply a foto)

Descrizione: Identifica l'invertebrato nella foto.

Esempio: [/identifyinvertebrate](#)

5 Risultati

5.1 Calibrazione

Nella Figura 7 è mostrato il sistema pronto per la sua autocalibrazione, l'utente ha inserito i codici ArUco nei vasi per la loro identificazione. Nella Figura 8 e nella Figura 9 sono mostrati due esempi di calibrazione: nel primo caso la procedura è andata a buon fine, mentre nel secondo la calibrazione risulta non corretta. In caso di calibrazione andata male vengono elencati i vasi non rilevati, in modo che l'utente possa capire dove intervenire.



Figura 7: Sistema pronto alla calibrazione



Figura 8: Esempio di calibrazione andata a buon fine

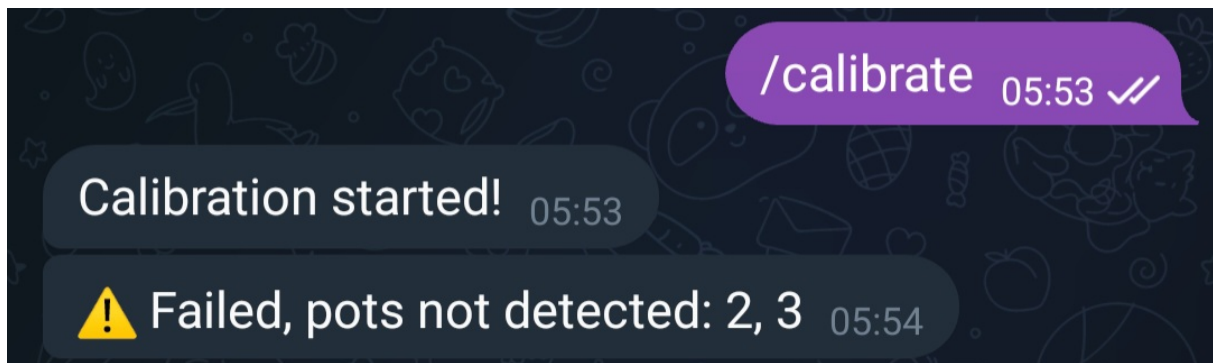


Figura 9: Esempio di calibrazione non riuscita

5.2 Configurazione

Nella Figura 10 e nella Figura 11 sono mostrati rispettivamente esempi di settaggio di parametri e visualizzazione della configurazione. Nell'immagine è inoltre presente la notifica relativa al serbatoio vuoto.

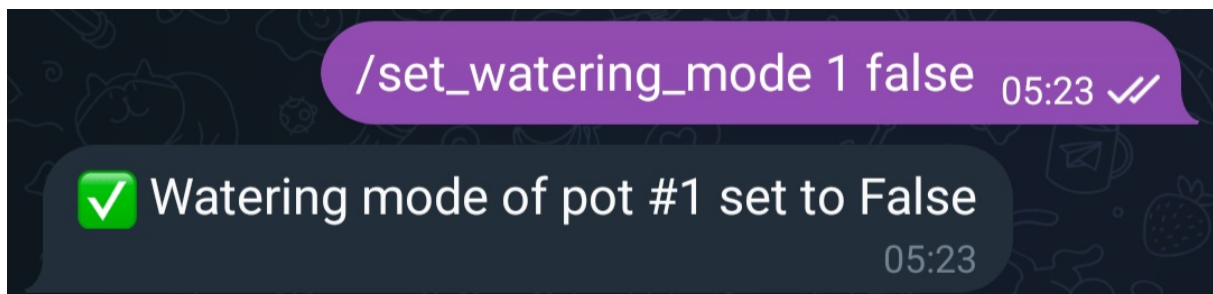


Figura 10: Esempio di configurazione di un parametro

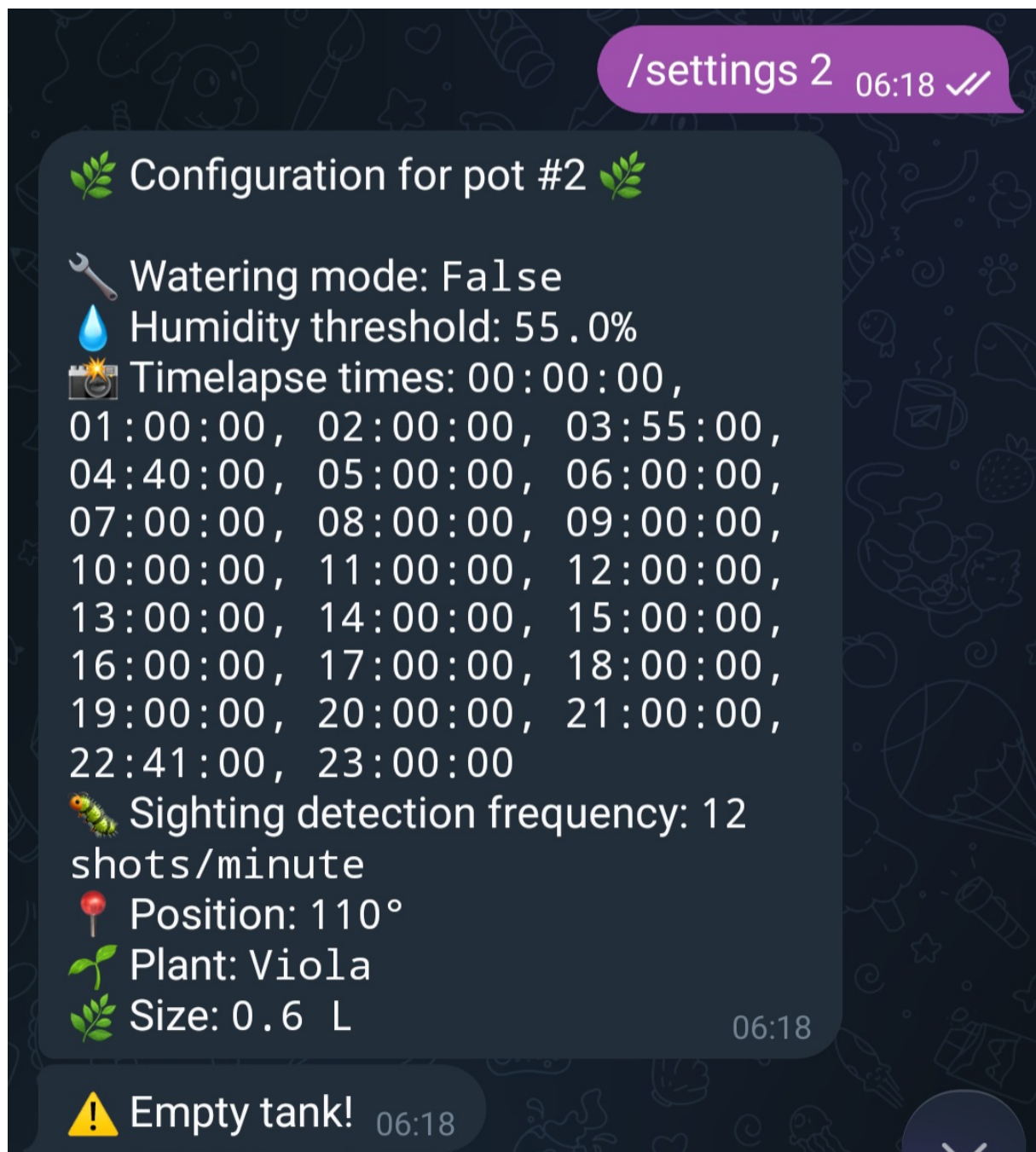


Figura 11: Esempio di visualizzazione della configurazione di un vaso

5.3 Timelapse

Nella Figura 12 è mostrata una sequenza di immagini di un timelapse generato, nella Figura 13 l'interazione con il bot che mostra la generazione di due timelapse, uno senza filtri e uno in bianco e nero.



Figura 12: Sequenza di immagini di un timelapse

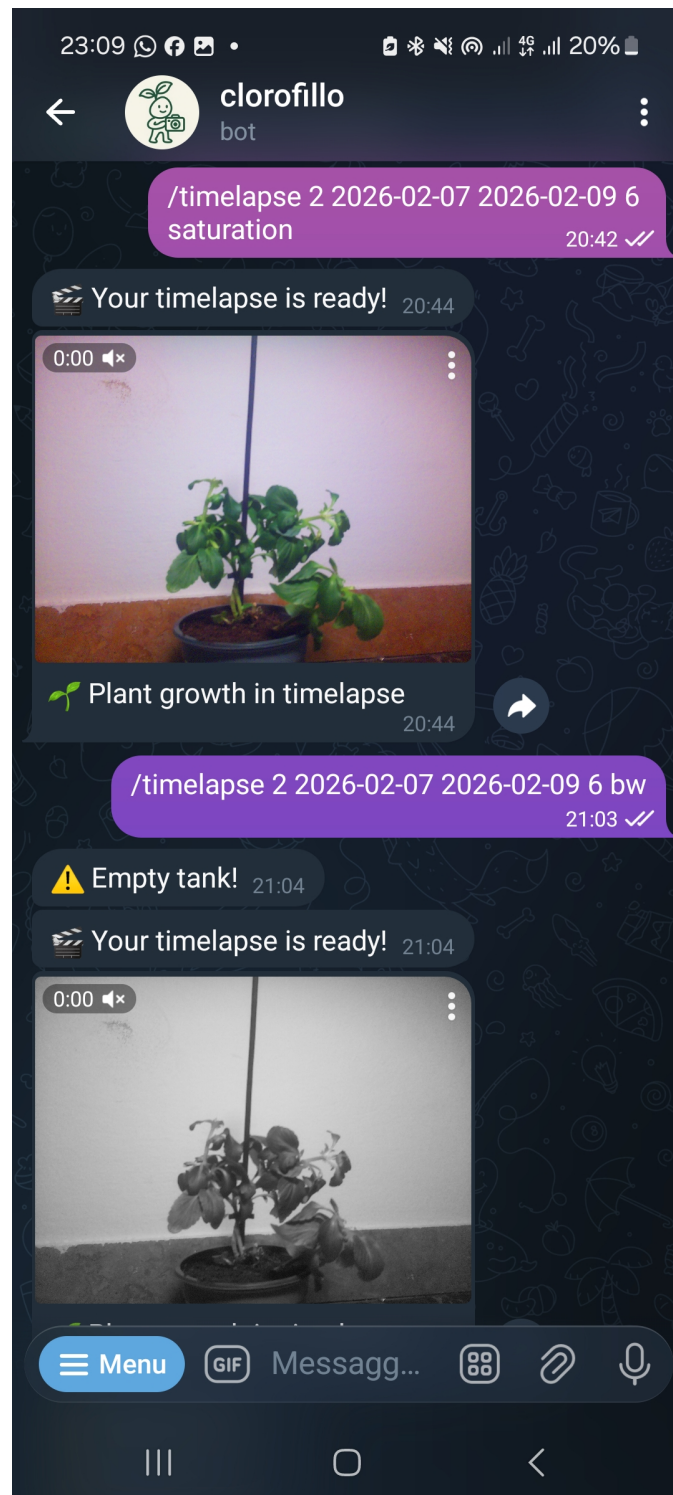


Figura 13: Generazione di un timelapse

5.4 Rilevamento corpi estranei

Nella Figura 14 e Figura 15 sono mostrati esempi di rilevamento di corpi estranei, in questo caso una mosca e un lombrico sulle foglie delle piante. In seguito nella Figura 16 e nella Figura 17 sono mostrati dei diari degli avvistamenti, generati tramite opportuno comando grazie al bot. Nella Figura 18 è mostrata l'interazione con il bot per identificare

un avvistamento.

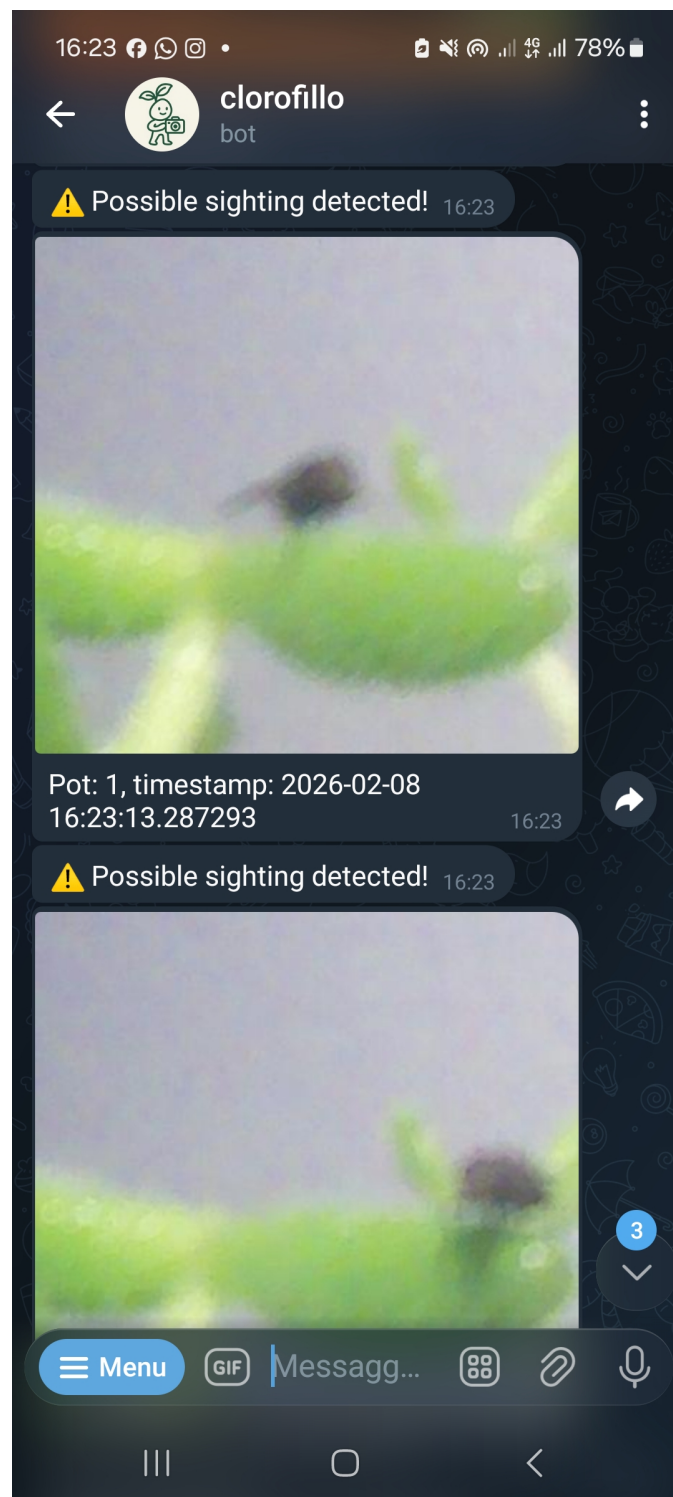


Figura 14: Esempio di rilevamento di una mosca

■ Sighting Diary - Pot 1

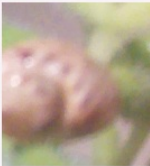
	Unknown ■ 08/02/2026
	Fly ■ 08/02/2026
	Snail Test ■ 17/01/2026
	Snail Test ■ 17/01/2026
	Mealworm Beetle ■ 10/02/2026
	Unknown ■ 10/02/2026

Figura 16: Diario degli avvistamenti

■ Sighting Diary - Pot 2

	Earthworm ■ 17/01/2026
	Cockroach ■ 17/01/2026
	Test ■ 17/01/2026
	Test ■ 10/02/2026

Figura 17: Diario degli avvistamenti



Figura 18: Identificazione insetto

Per effettuare alcuni test, come si vede dalle pagine dei diari degli avvistamenti, sono stati usati dei piccoli oggetti per simulare insetti, con lo scopo di vedere come il sistema reagisse con la presenza di corpi estranei. Venivano posati e poi rimossi dalle foglie, il sistema li ha rilevati correttamente.

Riguardo ai casi in cui gli insetti sono venuti a contatto con le piante e sono stati rilevati, prendiamo come esempio il caso di rilevamento di una mosca, le cui patch relative all'avvistamento sono riportate in Figura 19. La mosca svolazzava e si posava sulle foglie della pianta; il sistema, in più momenti, ha rilevato l'insetto e ha inviato via bot le relative patch. Si nota che l'interazione degli insetti con la pianta genera inevitabilmente anche falsi positivi: quando l'insetto si muove e abbandona una posizione, quella stessa posizione, ormai priva dell'insetto, viene comunque notificata, poiché rilevata come differente rispetto allo stato precedente in cui l'insetto era presente.

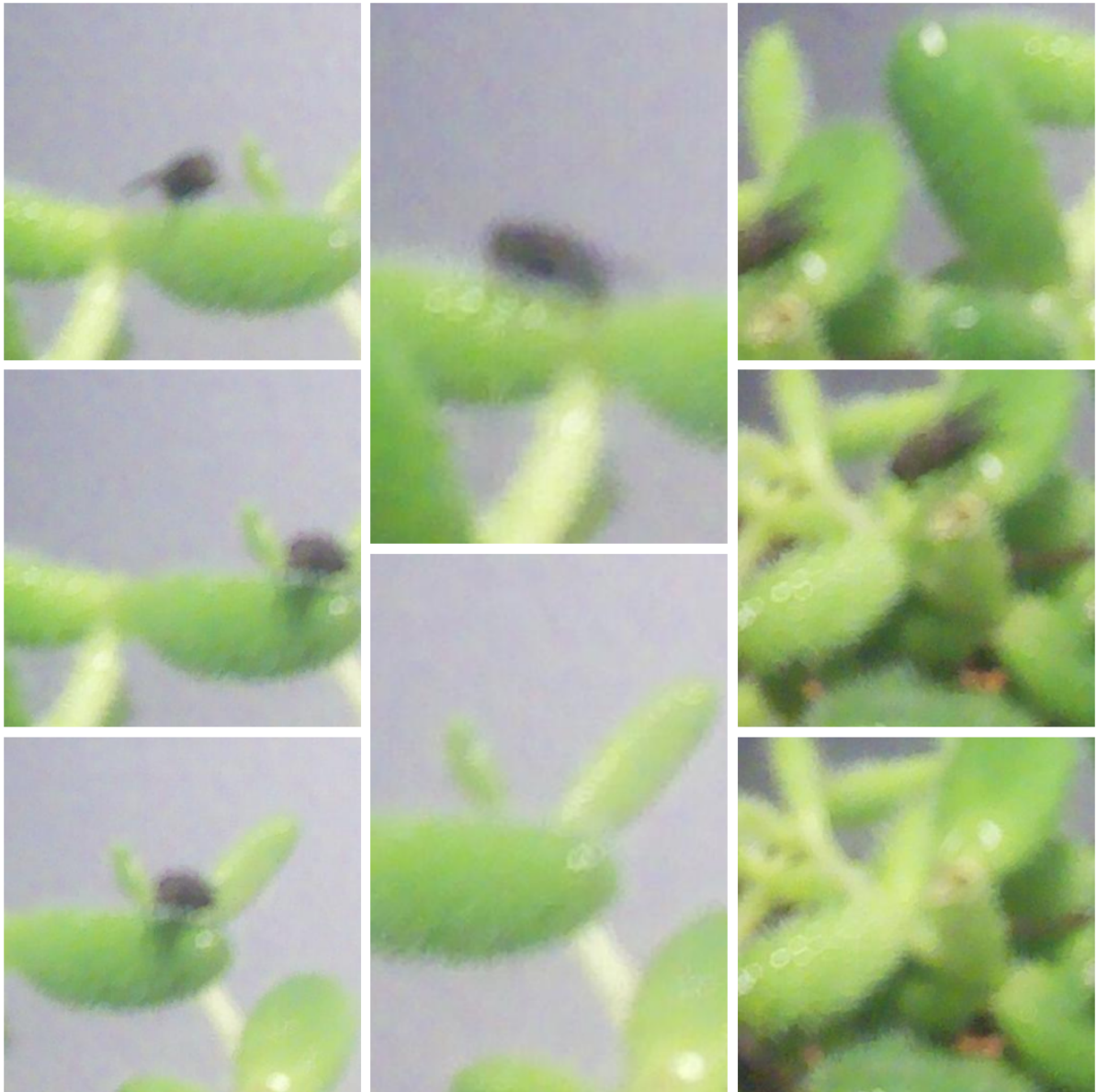


Figura 19: Avvistamenti di una mosca con relativi falsi positivi

6 Conclusioni

Il sistema soddisfa molto bene le funzionalità di irrigazione automatica e di timelapse. Il rilevamento di corpi estranei è la parte che necessita di qualche miglioramento. Lato hardware, bisognerebbe utilizzare una fotocamera più performante e fissarla sul braccio del servo motore in maniera più stabile, in modo da ottenere foto di migliore qualità (che permettano all'utente o a un servizio di identificazione esterno di riconoscere correttamente il soggetto) ed evitare falsi positivi causati dal disallineamento delle immagini. Lato software, l'algoritmo di rilevamento potrebbe essere migliorato rendendolo meno sensibile ai cambiamenti di luce e ai piccoli spostamenti delle piante. Una possibile strategia potrebbe essere quella di concentrare maggiormente risorse e sviluppo sulla parte visiva (timelapse e rilevamento dei corpi estranei), ottimizzando sia l'hardware sia il software per ottenere risultati più affidabili e precisi.